EY

Ernst & Young

GenAI RAG System

High-Level Architecture Documentation

Retrieval-Augmented Generation Platform — EY Internship 2026

Author	Likhith Diwakar
Organisation	EY (Ernst & Young)
Project	Internship GenAI Project
Document Type	High-Level Design (HLD) — Updated
Date	April 2026
Live URL	https://genai-rag-system-3.onrender.com

1. Executive Summary

The GenAI RAG System is a Retrieval-Augmented Generation (RAG) platform built during an EY internship. It allows users to ask natural-language questions about documents stored in Google Drive and receive accurate, cited answers grounded in the actual content of those documents — not generated from model memory.

The platform handles the full lifecycle autonomously: documents are ingested and indexed daily, queries are intelligently routed to the right retrieval and computation path, and responses are attributed back to their source files. The system has since been extended beyond the core RAG engine into a platform, incorporating a KnowledgeVerse dashboard, user interaction analytics, real-time document search, and behavioral data tracking.

Attribute	Value
Deployment Platform	Render Cloud (Docker containers)
Live URL	https://genai-rag-system-3.onrender.com
Supported Formats	PDF (text, scanned, chart-heavy, tables), DOCX, CSV
Retrieval Approach	Hybrid semantic + lexical search with intelligent re-ranking
LLM Strategy	Dynamic routing across three models based on query type
Ingestion Automation	Daily scheduled pipeline via GitHub Actions
Backup & Recovery	Full index restoration from Google Drive on every startup
Orchestration Model	Graph-based execution with conditional routing (LangGraph)
Dashboard (NEW)	KnowledgeVerse — Latest Docs, Frequently Visited, Recent Activity
Search (NEW)	Real-time prefix search with live dropdown suggestions
Analytics (NEW)	User behavior tracking, click logging, query history
Chat History (NEW)	Grouped by date, persistent across sessions

1.1 Platform — Landing Page

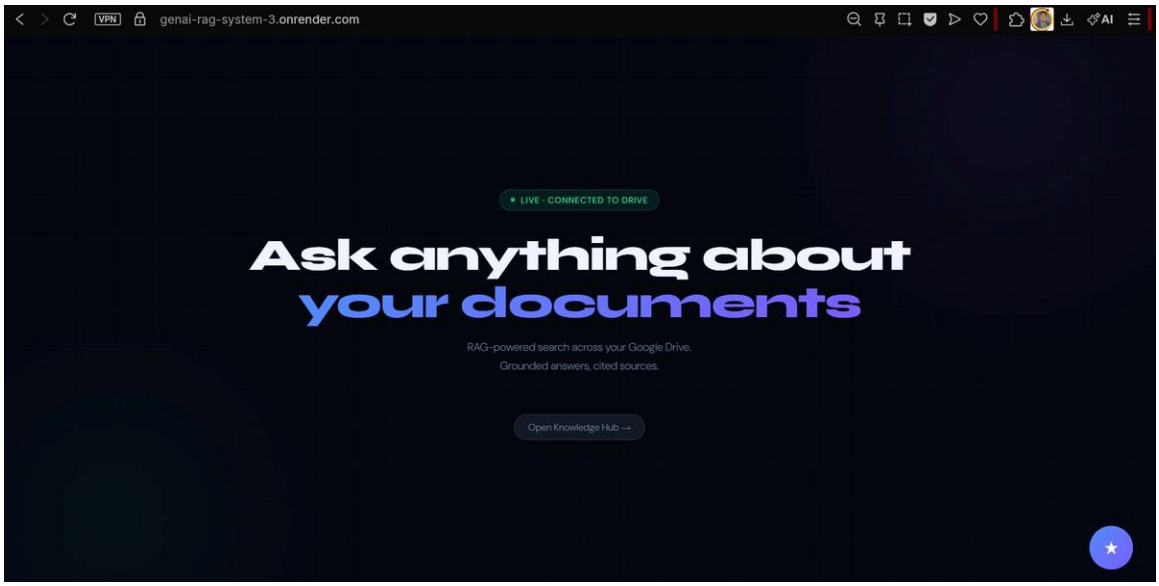

Figure 1: Live deployment at genai-rag-system-3.onrender.com

1.2 Platform — KnowledgeVerse Dashboard

NEW FEATURE

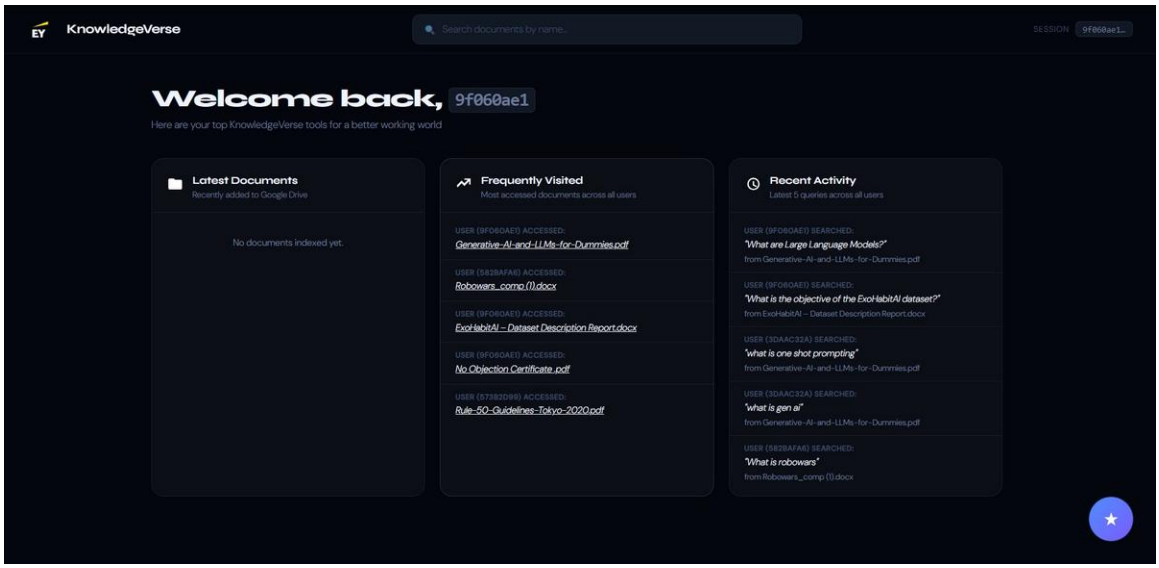

Figure 2: KnowledgeVerse dashboard showing Latest Documents, Frequently Visited, and Recent Activity panels

1.3 Platform — RAG Chat Interface with Response Caching

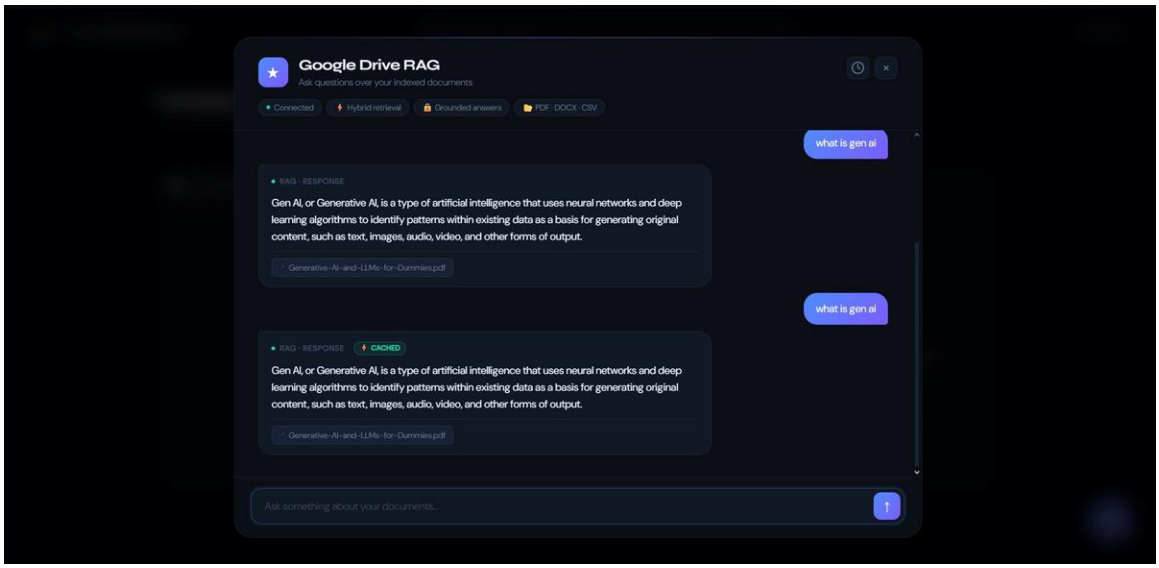

Figure 3: Google Drive RAG assistant showing grounded answer with source attribution and CACHED response indicator

1.4 Platform — Chat History with Date Grouping

NEW FEATURE

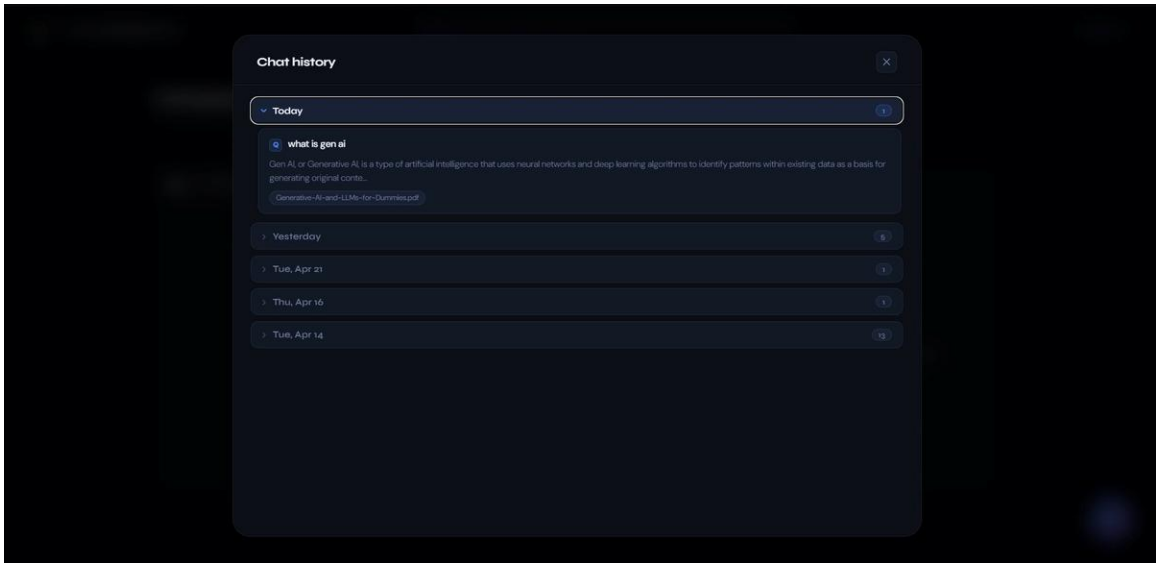

Figure 4: Chat history panel showing queries grouped by date with source document attribution

2. Project Background & Evolution

The system was developed iteratively across six phases between January and April 2026. The project started as a simple local prototype and was progressively hardened into a cloud platform, and then further extended into a website with a dashboard, analytics, and user interaction layers.

Phase	Period	Theme	What Changed
1 — Foundation	Jan 20 – Jan 27	Proof of Concept	Basic document ingestion from Google Drive into a local vector store; Streamlit chat interface; local LLM (Ollama/Mistral) for answer generation.
2 — Retrieval Quality	Feb 4 – Feb 14	Accuracy & Format Breadth	Switched to paragraph-level chunking; stronger embeddings; added PDF and DOCX support; introduced a deterministic computation path for CSV numerical queries.
3 — Cloud Migration	Feb 27	Scalability	Moved embeddings and LLMs to hosted cloud APIs; added Gemini Vision OCR for scanned and chart-heavy PDFs; modularised the codebase.
4 — Production Deploy	Mar 5	Reliability	Deployed to Render via Docker; migrated to Qdrant Cloud; replaced local scheduler with GitHub Actions; introduced two-layer backup and zero-downtime startup recovery.
5 — Advanced Retrieval	Mar – Apr 2026	Precision & Resilience	Added query rewriting, cross-encoder reranking, graph-based execution (LangGraph), multi-model LLM routing, and migrated frontend to React.js with FastAPI backend.
6 — Additional Enhancements	**Apr 2026**	**Full Product Platform**	**Added KnowledgeVerse dashboard, real-time search, user activity tracking, click event logging, chat history, response caching, and Google Drive URL integration.**

3. High-Level Architecture

The system is structured around six independently operating layers. The first five layers handle the core RAG engine; the sixth — the User Interaction & Analytics Layer — represents the expansion built on top of the RAG foundation.

3.1 Layer Overview

Layer	Purpose	Outcome
Data Sources & Ingestion	Discovers documents in Google Drive, extracts their content by format, chunks the text semantically, and writes embeddings to the vector store.	A fresh, up-to-date index of all documents — updated every night without manual intervention.
Storage	Holds all vector embeddings for semantic search (Qdrant Cloud) and all structured metadata and CSV data for deterministic computation (SQLite). Extended to also store latest documents, access logs, and user activity.	Fast, reliable retrieval of both unstructured document content and structured numerical data — plus user analytics.
Query & Retrieval	Classifies each incoming query, routes it to the right execution path (semantic or structured), retrieves the most relevant content, and generates a grounded answer via the appropriate LLM.	A cited, accurate answer that is traceable back to a specific source document.
Frontend & API	Presents the KnowledgeVerse dashboard and conversational chat interface to users (React.js) and exposes REST endpoints including new analytics endpoints (FastAPI).	An accessible, responsive product interface that any user can operate without technical knowledge.
Backup & Recovery	Snapshots the vector index and metadata database to Google Drive after every ingestion cycle and restores them automatically on every deployment startup.	The system is always query-ready within seconds of boot — even after a complete environment reset.
User Interaction & Analytics (NEW)	Tracks document access events, query history, and user sessions. Powers the dashboard's Frequently Visited and Recent Activity panels.	A behavioral data layer that provides visibility into how users interact with the document corpus across all sessions.

4. System Capabilities

The platform supports two categories of AI-powered user queries, each handled by a distinct execution path. Additionally, the system now exposes a full dashboard and interaction layer that enables document discovery, user navigation, and behavioral analytics.

4.1 Document Question Answering (Semantic Path)

When a user asks an open-ended or descriptive question about the contents of their documents, the system retrieves the most relevant passages using a hybrid search approach that combines vector similarity with keyword matching. Every response identifies the source document it drew from.

4.2 Structured Data Computation (Structured Path)

When a user asks a numerical or aggregation question targeting a CSV file, the system executes a deterministic computation directly against the stored data. No language model is involved in the computation itself; an LLM is used only to wrap the numeric result in a readable sentence.

4.3 KnowledgeVerse Dashboard

NEW FEATURE

The KnowledgeVerse dashboard is the primary entry point into the platform. It presents three live panels driven by real system data, giving users immediate visibility into the document corpus and community activity.

Panel	Data Source	Behaviour
Latest Documents	SQLite ingestion timestamps	Shows the 5 most recently ingested files with clickable Google Drive links. Auto-refreshes after ingestion cycles. Shared across all users.
Frequently Visited	Click event log in SQLite	Shows the most-accessed documents across all users, listing each access event with the user session ID and a clickable document link.
Recent Activity	Query log in SQLite	Shows the latest 5 queries across all users, displaying query text, source document, and session ID.

4.4 Real-Time Document Search

NEW FEATURE

A prefix-based search bar in the dashboard header enables real-time document discovery. Users type a partial file name and receive live dropdown suggestions filtered against the indexed document corpus. Selecting a result opens the document directly in Google Drive.

Feature	Detail
Search mechanism	Prefix-based match against file names stored in SQLite
Suggestions	Dropdown renders dynamically on each keystroke
Click action	Opens corresponding Google Drive file in a new tab
API endpoint	/search-documents — returns matching file names and Drive URLs

4.5 Chat History

NEW FEATURE

The RAG chat interface now surfaces full query history, grouped by date. Each history entry shows the query text, the response summary, and the source document cited. History is persisted across sessions and grouped into Today, Yesterday, and prior dated sections.

4.6 Response Caching

NEW FEATURE

Repeated queries return a CACHED response instantly, bypassing the full retrieval and LLM pipeline. Cached responses are visually distinguished in the chat UI with a CACHED badge, improving perceived performance for repeated questions.

4.7 Supported Document Types

Format	How It Is Processed	Queries It Supports
PDF — text-based	Text extracted page by page and chunked into semantically coherent passages.	Any natural-language question about the document's content.
PDF — scanned / charts	Pages are processed through vision-based OCR to extract text from images and diagrams.	Questions about scanned documents, charts, or image-heavy reports.
PDF — embedded tables	Table rows are extracted and preserved as structured chunks with row-level integrity.	Questions about tabular data within PDF documents.
DOCX	Paragraphs and headings are extracted preserving the document's structural hierarchy.	Any natural-language question about the document's content.
CSV	Rows and columns are stored directly for deterministic computation.	Numerical aggregation queries: sums, averages, counts, comparisons, filters.

5. End-to-End Data Flow

The system operates three distinct data flows: an ingestion flow that runs nightly, a query flow that executes in real time, and an interaction tracking flow that records all user events asynchronously.

5.1 Ingestion Flow (Nightly)

Step	What Happens
1. Trigger	GitHub Actions fires the ingestion workflow automatically at 3:00 AM IST every day.
2. Discovery	The system connects to Google Drive and identifies documents that are new or have been modified since the last run. Unchanged documents are skipped.
3. Extraction	Each document is routed to the appropriate parser based on its format.
4. Chunking	Extracted text is split into semantically meaningful segments using format-appropriate strategies.
5. Embedding	Each chunk is converted into a 384-dimensional numerical vector using a hosted embedding model.
6. Storage	Vector embeddings are written to Qdrant Cloud. Structured CSV data is written to SQLite.
7. Latest Docs Update (NEW)	The SQLite tracker's latest_documents table is updated with the ingestion timestamp of each new file, making them immediately visible in the dashboard.
8. Backup	The SQLite database is compressed and uploaded to Google Drive. Qdrant creates a collection snapshot alongside.

5.2 Query Flow (Real Time)

Step	What Happens
1. User Query	The user types a question in the React.js chat interface. The frontend sends the query to the FastAPI backend.
2. Classification	The backend classifies the query as structured (CSV) or semantic (document content).
3a. Structured Path	If structured: Pandas computes the result directly from SQLite. Result is exact and deterministic.
3b. Semantic Path	If semantic: the query is expanded, then passed through hybrid search (dense + BM25) against Qdrant Cloud. Retrieved passages are re-ranked before being passed to an LLM.
4. LLM Generation	The appropriate language model is selected and generates a grounded answer from the retrieved context.
5. Response	The answer is returned to the frontend with the source document name.
6. Cache Store (NEW)	The query and response are stored in the cache. Subsequent identical queries return the cached result with a CACHED badge, bypassing retrieval.
7. Activity Log (NEW)	The query text, source document, and session ID are logged to the SQLite activity table, making them visible in the dashboard's Recent Activity panel.

5.3 Interaction Tracking Flow (Asynchronous)

NEW FLOW

Event	Trigger	Data Recorded	Dashboard Impact
Document Click	User clicks a document link in dashboard or search results	Session ID, file name, file ID, timestamp	Increments Frequently Visited count
Query Submitted	User submits a question in the chat interface	Session ID, query text, source document, timestamp	Appears in Recent Activity panel
Search Used	User types in the search bar and selects a suggestion	Session ID, search term, selected document	Reflected in access logs

6. API Extensions

NEW SECTION

The FastAPI backend has been extended with additional endpoints to support the dashboard, search, and analytics features. These endpoints complement the existing query processing endpoint.

Endpoint	Method	Purpose	Returns
/query	POST	Core RAG query endpoint	Answer, source document, cache status
/latest-documents (NEW)	GET	Returns the 5 most recently ingested documents	File names, Drive URLs, ingestion timestamps
/search-documents (NEW)	GET ?q={prefix}	Prefix-based document name search	Matching file names and Google Drive URLs
/track-access (NEW)	POST	Logs a document click event for a user session	Acknowledgement
/recent-activity (NEW)	GET	Returns latest 5 queries across all users	Query text, source document, session ID
/frequently-visited (NEW)	GET	Returns most-accessed documents across all users	File names, access counts, session IDs

7. Key Design Decisions

The following decisions define how the platform behaves, where it invests its complexity, and what guarantees it offers to users.

Decision	Rationale
Hybrid retrieval over pure vector search	Combining semantic vector search with BM25 captures both conceptual similarity and exact term matches, handling paraphrased questions and precise technical terms simultaneously.
Deterministic path for numerical queries	Running numerical queries through a computation engine rather than an LLM eliminates hallucination entirely for structured data, guaranteeing reproducible results.
Query rewriting before retrieval	Short or vague queries often fail to match source document vocabulary. Expanding the query before retrieval improves recall without requiring users to reformulate.
Post-retrieval re-ranking	Retrieving a large candidate pool and re-ranking by relevance produces better final selections than retrieving fewer results initially.
Graph-based query orchestration	Implementing the query pipeline as a directed graph (LangGraph) allows conditional branching, modular node design, and clean state management.
Two-layer backup and boot-time recovery	Storing both the vector index snapshot and the metadata database on Google Drive and restoring them at startup ensures the system is always fully operational within seconds of boot.
Multi-model LLM routing	No single model is optimal for all query types. Routing general QA, summarisation, and complex reasoning to different models optimises both quality and cost.
Global shared state for analytics (NEW)	Dashboard panels (Frequently Visited, Recent Activity) operate on shared global state rather than per-user state. This surfaces cross-user usage patterns and creates a community-visible activity feed.
SQLite extended as analytics store (NEW)	Rather than introducing a separate analytics database, the existing SQLite tracker was extended with access log and activity tables. This keeps the system single-database and simplifies the backup strategy.
Response caching at query level (NEW)	Caching identical query-response pairs avoids redundant retrieval and LLM calls, reducing latency and API cost for repeated questions without sacrificing accuracy.

8. Technology Summary

Area	Technology	Role in the System
Document Ingestion	GitHub Actions	Scheduled daily pipeline that triggers document discovery, extraction, and indexing.
Text Extraction	pdfplumber, python-docx, Pandas	Format-specific parsers for extracting clean text from PDF, DOCX, and CSV files.
Vision OCR	Gemini Vision Flash	Converts scanned pages and chart-heavy PDFs into extractable text using a multimodal vision model.
Embeddings	BAAI/bge-small-en-v1.5 via HuggingFace	Generates 384-dimensional semantic vectors for all document chunks and incoming queries.
Vector Store	Qdrant Cloud	Hosts and queries all document embeddings with approximate nearest-neighbour search.
Structured Store	SQLite (tracker.db)	Stores document metadata, CSV data, access logs, activity logs, and latest document records.
Lexical Search	BM25	Keyword-based retrieval index rebuilt at startup for exact and near-exact query term matching.
Re-ranking	ms-marco-MiniLM cross-encoder	Scores each retrieved passage against the query to reorder results by true relevance.
Query Orchestration	LangGraph	Manages the graph-based query execution pipeline with conditional routing and shared state.
Primary LLM	Llama 3.3 70B (Groq)	Handles general question answering — the primary model in the LLM router.
Summarisation LLM	GPT-4o Mini (OpenRouter)	Handles queries that require condensed summaries of longer document content.
Reasoning LLM	Claude Haiku (OpenRouter)	Handles complex reasoning queries that benefit from a higher-capability model.
Backend API	FastAPI (Render)	Exposes REST endpoints for the chat interface, dashboard, search, and analytics.
Frontend	React.js (Render)	Single-page application with KnowledgeVerse dashboard, chat interface, search bar, and source attribution.
Containerisation	Docker	Packages the full application into a reproducible container for consistent deployment.
Response Cache (NEW)	In-memory / SQLite cache	Stores query-response pairs to serve repeated questions instantly with a CACHED indicator.

Area	Technology	Role in the System
Analytics Store (NEW)	SQLite extended tables	Records document access events and query logs to power dashboard analytics panels.

9. Deployment Overview

The system is split across two separate deployment targets. The serving application requires a persistent, always-on runtime, while the ingestion job is a scheduled batch process that does not need to share infrastructure with the live platform.

9.1 Runtime — Render (Docker)

The full platform — including the React.js frontend, the FastAPI backend, the query engine, the dashboard APIs, and the Qdrant connection — runs inside a Docker container hosted on Render Cloud. On every cold start, the application automatically retrieves the latest SQLite database and Qdrant snapshot from Google Drive and restores the full document index before serving any requests.

9.2 Ingestion — GitHub Actions

Document ingestion runs as a scheduled GitHub Actions workflow at 3:00 AM IST daily. The workflow discovers new and modified files in Google Drive, processes them through the ingestion pipeline, updates the vector store and metadata database, updates the latest documents table in SQLite, and uploads fresh backup artefacts. The running application is not restarted; new documents appear in the dashboard automatically after indexing completes.

9.3 Ingestion → Dashboard Sync

NEW

A direct link now exists between the nightly ingestion pipeline and the dashboard UI. After each ingestion cycle completes, the SQLite latest_documents table is updated with fresh ingestion timestamps. The next dashboard load automatically reflects the newly ingested files in the Latest Documents panel, with no manual intervention required.

10. Constraints & Future Direction

Area	Current State	Future Direction
Observability	Application logs are written locally within the container. No centralised log aggregation or request tracing.	Implement structured logging, distributed tracing, and a monitoring dashboard to enable measurement and debugging.
Query Latency	End-to-end latency is acceptable for current usage scale but has not been profiled systematically.	Profile each pipeline stage and optimise embedding, retrieval, and LLM routing latencies.
External Integrations	The FastAPI backend exposes basic endpoints. No external-facing API gateway for enterprise integration.	Build a versioned API gateway with authentication to enable webhook-based integrations and programmatic document querying.
Document Sources	All documents are sourced exclusively from Google Drive.	Evaluate Azure Blob Storage and SharePoint as alternative or additional document source layers.
Frontend Maturity	Core functionality covered. Error states, streaming responses, and accessibility are partially implemented.	Refine the interface with streaming response display, improved error handling, and full accessibility compliance.
Analytics Depth (NEW)	Activity and access tracking are operational at a basic level.	Use behavioral data to power document recommendations, auto-rank frequently accessed documents higher in retrieval, and surface usage analytics to system administrators.
Personalisation (NEW)	Dashboard shows global shared state for all users.	Introduce per-user preferences and personalised document recommendations based on individual query history.

11. Glossary

Term	Definition
RAG (Retrieval-Augmented Generation)	A technique that grounds a language model's responses in content retrieved from external documents, rather than relying on the model's parametric knowledge. Prevents hallucination and provides source traceability.
Vector Embedding	A numerical representation of a piece of text as a point in high-dimensional space. Texts with similar meaning are placed close together, enabling similarity-based retrieval.
Hybrid Retrieval	A retrieval strategy that combines vector similarity search (semantic) with keyword-based search (lexical) to capture both conceptual and exact-match relevance signals.
BM25	A keyword-based retrieval algorithm that scores documents based on the frequency and rarity of query terms. Effective for precise technical terminology.
Re-ranking	A second-pass scoring step applied after initial retrieval to reorder results by their true relevance to the query.
LangGraph	A library for building LLM pipelines as directed graphs with shared state, enabling modular, conditionally routed execution flows.
Qdrant	A cloud-native vector database designed for high-performance approximate nearest-neighbour search across large collections of embeddings.
LLM Router	A component that selects the most appropriate language model for each query based on its type (QA, summarisation, reasoning) and current model availability.
Vision OCR	Optical Character Recognition performed by a multimodal vision model. Used to extract text from scanned PDFs, charts, and image-embedded tables.
KnowledgeVerse (NEW)	The dashboard layer of the platform, providing document discovery, user activity feeds, and real-time search over the indexed document corpus.
Interaction Tracking (NEW)	The system's behavioral data layer — records document access events and query logs per user session to power the Frequently Visited and Recent Activity dashboard panels.
Response Cache (NEW)	A query-level cache that stores previously answered questions and their responses. Identical queries return cached results instantly, marked with a CACHED indicator in the UI.
HLD	High-Level Design — a document describing a system's architecture, major components, data flows, and design decisions at a conceptual level, without component-level implementation detail.